ShopSphere

E-Commerce Order Management REST API

A resume project built with **Java, Spring Boot, Spring Security (JWT), Spring Data JPA, and MySQL**. This document captures every file built so far — Steps 1 through 8 — along with the reasoning behind each design decision, so it can be revisited, explained in an interview, or used to pick the project back up later without losing context.

Base package: `com.shopsphere.shopsphere`

Covered so far: Project setup · Entities · Repositories · DTOs · JWT Security · Security Config · Auth Controller/Service · Category & Product REST APIs

Still ahead: Order placement & stock locking · Global exception handling · Testing

Table of Contents

- Step 1 — Project Setup (pom.xml, application.properties)
- Step 2 — Entities (Role, OrderStatus, User, Category, Product, Order, OrderItem)
- Step 3 — Repositories (Spring Data JPA interfaces)
- Step 4 — DTOs (Request & Response objects with validation)
- Step 5 — Security Core (JwtService, UserPrincipal, CustomUserDetailsService, JwtAuthenticationFilter)
- Step 6 — SecurityConfig (filter chain, password encoding, role rules)
- Step 7 — Auth Controller & Service (register / login)
- Step 8 — Category & Product REST APIs (CRUD, pagination, search)
- Verified Test Results (Postman)

Step 1 — Project Setup

Spring Boot project generated via Spring Initializr (Maven, Java 21, Spring Boot 3.x), with the JWT library added manually since it isn't offered on the Initializr.

pom.xml — key dependencies

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>

    <!-- Added manually: JWT support -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/shopsphere
spring.datasource.username=root
spring.datasource.password=YOUR_PASSWORD_HERE
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true

app.jwt.secret=replace-this-with-a-long-random-string-later
app.jwt.expiration-ms=3600000
```

Why this matters:

JJWT was added by hand because JSON Web Token signing/verification is a small, standard library, not a platform or managed service — it fits the "no external infra" constraint for this project.
spring.jpa.hibernate.ddl-auto=update means Hibernate auto-creates and updates tables directly from the `@Entity` classes, so no manual CREATE TABLE statements are needed for this project (in a real production system, a migration tool like Flyway would replace this). Credentials and the JWT secret live only in this local, git-ignored file — never hardcoded or committed.

Step 2 — Entities

Seven files: two enums (Role, OrderStatus) and five JPA entities mapping directly to the relational schema. Hibernate generates the actual SQL tables from these at startup.

entity/Role.java

```
package com.shopsphere.shopsphere.entity;

public enum Role {
    ROLE_CUSTOMER,
    ROLE_ADMIN
}
```

entity/OrderStatus.java

```
package com.shopsphere.shopsphere.entity;

public enum OrderStatus {
    PENDING,
    CONFIRMED,
    SHIPPED,
    DELIVERED,
    CANCELLED
}
```

entity/User.java

```
package com.shopsphere.shopsphere.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.Instant;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users", uniqueConstraints = @UniqueConstraint(columnNames = "email"))
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String fullName;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password; // stored as a BCrypt hash, never plain text

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
    @Enumerated(EnumType.STRING)
    @Column(name = "role")
    @Builder.Default
    private Set<Role> roles = new HashSet<>();

    @Column(nullable = false)
    @Builder.Default
    private boolean enabled = true;

    @Column(nullable = false, updatable = false)
    private Instant createdAt;
```

```
    @PrePersist
    protected void onCreate() {
        this.createdAt = Instant.now();
    }
}
```

Why this matters:

@ElementCollection for roles avoids over-engineering a full Role entity/table for what are really just labels. **@Builder.Default** is essential any time @Builder is combined with a field that has a default value — without it, Lombok silently gives you null instead of an empty HashSet. The password field is just a String at the entity level; hashing happens in the service layer (separation of concerns: entities hold data, they don't enforce security policy). @PrePersist auto-stamps createdAt without needing to remember to set it manually.

entity/Category.java

```
package com.shopsphere.shopsphere.entity;

import jakarta.persistence.*;
import lombok.*;

import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "categories")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    private String description;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true)
    @Builder.Default
    private Set<Product> products = new HashSet<>();
}
```

Why this matters:

mappedBy = "category" tells JPA that Product owns the actual foreign key column — Category only holds the inverse side for convenience, no extra column is created on the categories table. **cascade = ALL, orphanRemoval = true** means deleting a Category deletes its Products too — a deliberate simplification for this project; a real system might instead block deleting a category that still has products.

entity/Product.java

```
package com.shopsphere.shopsphere.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;
import java.time.Instant;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

```
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @Column(nullable = false)
    private Integer stockQuantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    @Column(nullable = false, updatable = false)
    private Instant createdAt;

    @Column(nullable = false)
    private Instant updatedAt;

    @Version
    private Long version;

    @PrePersist
    protected void onCreate() {
        this.createdAt = Instant.now();
        this.updatedAt = Instant.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = Instant.now();
    }
}
```

Why this matters:

BigDecimal, never double/float, for money. Floating point types can't represent decimal fractions like 19.99 exactly in binary, causing rounding errors when summing totals — a classic interview talking point. **@Version** enables JPA optimistic locking: Hibernate auto-increments this column on every update and throws an exception if two transactions try to update the same row from stale data. This is one half of the strategy (paired with a pessimistic lock at the repository level) used to prevent overselling stock during concurrent order placement in Step 9.

entity/Order.java

```
package com.shopsphere.shopsphere.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orders")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

```
@Builder
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    @Builder.Default
    private List<OrderItem> items = new ArrayList<>();

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal totalAmount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private OrderStatus status;

    private String shippingAddress;

    @Column(nullable = false, updatable = false)
    private Instant createdAt;

    private Instant updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = Instant.now();
        this.updatedAt = Instant.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = Instant.now();
    }
}
```

entity/OrderItem.java

```
package com.shopsphere.shopsphere.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;

@Entity
@Table(name = "order_items")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class OrderItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)
    private Integer quantity;
    @Column(nullable = false, precision = 10, scale = 2)
```

```
    private BigDecimal unitPriceAtPurchase;
}
```

Why this matters:

@Enumerated(EnumType.STRING) on status stores readable text ("PENDING", "SHIPPED"...) instead of the default ordinal (0, 1, 2...). Ordinal storage is a classic trap: reordering the enum later silently corrupts the meaning of every existing row. **unitPriceAtPurchase is a price snapshot, not a live reference** — this is the most interesting design decision in the schema. If OrderItem instead looked up product.getPrice() live, changing a product's price today would silently rewrite the totals on every past order that included it. Order history must reflect what the customer actually paid at the time of purchase.

Step 3 — Repositories

Thin interfaces extending Spring Data JPA's JpaRepository. Spring generates the actual implementation at runtime by parsing method names and annotations — no SQL written by hand.

repository/UserRepository.java

```
package com.shopsphere.shopsphere.repository;

import com.shopsphere.shopsphere.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

repository/CategoryRepository.java

```
package com.shopsphere.shopsphere.repository;

import com.shopsphere.shopsphere.entity.Category;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByNameIgnoreCase(String name);
    boolean existsByNameIgnoreCase(String name);
}
```

repository/ProductRepository.java

```
package com.shopsphere.shopsphere.repository;

import com.shopsphere.shopsphere.entity.Product;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import jakarta.persistence.LockModeType;
import java.util.Optional;

public interface ProductRepository extends JpaRepository<Product, Long> {

    Page<Product> findByCategoryId(Long categoryId, Pageable pageable);

    Page<Product> findByNameContainingIgnoreCase(String name, Pageable pageable);

    // Pessimistic lock used at checkout time to serialize stock decrements
    // and prevent overselling under concurrent order placement.
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select p from Product p where p.id = :id")
    Optional<Product> findByIdForUpdate(@Param("id") Long id);
}
```

repository/OrderRepository.java

```
package com.shopsphere.shopsphere.repository;

import com.shopsphere.shopsphere.entity.Order;
import com.shopsphere.shopsphere.entity.OrderStatus;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {
```

```
    Page<Order> findByUserId(Long userId, Pageable pageable);
    Page<Order> findByStatus(OrderStatus status, Pageable pageable);
}
```

repository/OrderItemRepository.java

```
package com.shopsphere.shopsphere.repository;

import com.shopsphere.shopsphere.entity.OrderItem;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderItemRepository extends JpaRepository<OrderItem, Long> {
}
```

Why this matters:

Extending JpaRepository<Entity, IdType> alone gives save(), findById(), findAll(), delete(), and pagination with zero implementation code — Spring builds a proxy at startup by parsing the interface. Method-name query derivation (findByEmail, ContainingIgnoreCase → case-insensitive LIKE) works for simple lookups; @Query with JPQL is used once logic gets more complex, as in findByIdForUpdate. **@Lock(PESSIMISTIC_WRITE)** is the second half of the overselling-prevention strategy (alongside @Version on Product): fetching a product this way places a row-level database lock, forcing any concurrent transaction touching the same row to wait until this one commits or rolls back. Page<T> and Pageable give pagination/sorting almost for free — e.g. GET /api/products?page=0&size=20&sort=price,desc — without manual LIMIT/OFFSET SQL.

Step 4 — DTOs (Data Transfer Objects)

DTOs decouple the API's public shape from the database schema. Returning entities directly would leak the password hash and risk Hibernate lazy-loading serialization issues on relationships. Split into dto/request (validated input) and dto/response (safe output).

Request DTOs

dto/request/RegisterRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class RegisterRequest {

    @NotBlank(message = "Full name is required")
    private String fullName;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;
}
```

dto/request/LoginRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
public class LoginRequest {

    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String password;
}
```

dto/request/CategoryRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
public class CategoryRequest {

    @NotBlank(message = "Category name is required")
    private String name;

    private String description;
}
```

dto/request/ProductRequest.java

```
package com.shopsphere.shopsphere.dto.request;
```

```
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

import java.math.BigDecimal;

@Data
public class ProductRequest {

    @NotBlank(message = "Product name is required")
    private String name;

    private String description;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01", message = "Price must be greater than 0")
    private BigDecimal price;

    @NotNull(message = "Stock quantity is required")
    @Min(value = 0, message = "Stock quantity cannot be negative")
    private Integer stockQuantity;

    @NotNull(message = "Category id is required")
    private Long categoryId;
}
```

dto/request/OrderItemRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

@Data
public class OrderItemRequest {

    @NotNull(message = "Product id is required")
    private Long productId;

    @NotNull(message = "Quantity is required")
    @Min(value = 1, message = "Quantity must be at least 1")
    private Integer quantity;
}
```

dto/request/CreateOrderRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import lombok.Data;

import java.util.List;

@Data
public class CreateOrderRequest {

    @NotEmpty(message = "Order must contain at least one item")
    @Valid
    private List<OrderItemRequest> items;

    @NotBlank(message = "Shipping address is required")
    private String shippingAddress;
}
```

dto/request/UpdateOrderStatusRequest.java

```
package com.shopsphere.shopsphere.dto.request;

import com.shopsphere.shopsphere.entity.OrderStatus;
```

```
import jakarta.validation.constraints.NotNull;
import lombok.Data;

@Data
public class UpdateOrderStatusRequest {

    @NotNull(message = "Status is required")
    private OrderStatus status;
}
```

Why this matters:

@Valid on the items list in CreateOrderRequest is essential: Bean Validation does not automatically cascade into nested objects or list elements. Without it, the @NotNull/@Min annotations inside OrderItemRequest would silently never run when nested inside another object — a genuinely common real bug. Rule of thumb: any DTO field that is itself a validated object (or list of them) needs its own @Valid.

Response DTOs

dto/response/AuthResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class AuthResponse {
    private String accessToken;
    private String tokenType;
    private Long expiresInMs;
}
```

dto/response/CategoryResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CategoryResponse {
    private Long id;
    private String name;
    private String description;
}
```

dto/response/ProductResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;

@Data
@Builder
@NoArgsConstructor
```

```
@AllArgsConstructor
public class ProductResponse {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stockQuantity;
    private Long categoryId;
    private String categoryName;
}
```

dto/response/OrderItemResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class OrderItemResponse {
    private Long productId;
    private String productName;
    private Integer quantity;
    private BigDecimal unitPriceAtPurchase;
    private BigDecimal lineTotal;
}
```

dto/response/OrderResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import com.shopsphere.shopsphere.entity.OrderStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class OrderResponse {
    private Long id;
    private Long userId;
    private List<OrderItemResponse> items;
    private BigDecimal totalAmount;
    private OrderStatus status;
    private String shippingAddress;
    private Instant createdAt;
}
```

dto/response/ApiErrorResponse.java

```
package com.shopsphere.shopsphere.dto.response;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.Instant;
import java.util.List;
@Data
```

```
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ApiErrorResponse {
    private Instant timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private List<String> details;
}
```

Why this matters:

AuthResponse intentionally carries a single access token (no refresh-token rotation) — this keeps auth simple and easy to reason about/test, and is still a completely legitimate, resume-worthy pattern. ApiErrorResponse is defined now but not wired up yet — it will be used by the global exception handler (a later step) to give every error a consistent JSON shape instead of Spring's default error page.

Step 5 — Security Core (JWT)

JWT authentication: the user logs in once and receives a signed token containing their identity. The client sends it on every future request via the Authorization header; the server verifies the signature and extracts identity with no server-side session and no database lookup needed on every request. This is what makes JWT auth stateless and scalable.

security/JwtService.java

```
package com.shopsphere.shopsphere.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey signingKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(UserDetails userDetails) {
        Date now = new Date();
        return Jwts.builder()
                .claims(new HashMap<>())
                .subject(userDetails.getUsername())
                .issuedAt(now)
                .expiration(new Date(now.getTime() + expirationMs))
                .signWith(signingKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    public long getExpirationMs() {
        return expirationMs;
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }

    private <T> T extractClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parser()
                .verifyWith(signingKey())
                .build()
```

```
                .parseSignedClaims(token)
                .getPayload();
        return resolver.apply(claims);
    }
}
```

security/UserPrincipal.java

```
package com.shopsphere.shopsphere.security;

import com.shopsphere.shopsphere.entity.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.stream.Collectors;

public class UserPrincipal implements UserDetails {

    private final User user;

    public UserPrincipal(User user) {
        this.user = user;
    }

    public Long getId() {
        return user.getId();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority(role.name()))
                .collect(Collectors.toList());
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked() { return true; }

    @Override
    public boolean isCredentialsNonExpired() { return true; }

    @Override
    public boolean isEnabled() {
        return user.isEnabled();
    }
}
```

security/CustomUserDetailsService.java

```
package com.shopsphere.shopsphere.security;

import com.shopsphere.shopsphere.entity.User;
import com.shopsphere.shopsphere.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
@Service
```

```
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
                .orElseThrow(() -> new UsernameNotFoundException("No user found with email: " + email));
        return new UserPrincipal(user);
    }
}
```

security/JwtAuthenticationFilter.java

```
package com.shopsphere.shopsphere.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain filterChain) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(7);
        final String userEmail;

        try {
            userEmail = jwtService.extractUsername(jwt);
        } catch (Exception e) {
            filterChain.doFilter(request, response);
            return;
        }

        if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userDetailsService.loadUserByUsername(userEmail);

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Why this matters:

JwtService pulls its secret and expiry from application.properties via @Value, then builds/validates HS256-signed tokens with the user's email as the subject claim. **UserPrincipal** is an adapter: Spring Security only understands its own UserDetails interface, so this class translates the custom User entity into username/password/authorities that Spring understands, prefixing roles as ROLE_X per Spring Security's convention.
CustomUserDetailsService is the bridge between Spring Security's internal machinery and the actual UserRepository lookup. **JwtAuthenticationFilter** is the real gatekeeper: it runs on every incoming request, checks for a Bearer token, validates it, and if valid, manually populates the SecurityContextHolder so the rest of the request is treated as authenticated. If no token or an invalid one is present, the request continues unauthenticated and SecurityConfig's access rules decide what happens next.

Step 6 — SecurityConfig

The file that ties everything together: password encoding, the authentication provider, and endpoint-level access rules by role and HTTP method.

config/SecurityConfig.java

```
package com.shopsphere.shopsphere.config;

import com.shopsphere.shopsphere.security.CustomUserDetailsService;
import com.shopsphere.shopsphere.security.JwtAuthenticationFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final CustomUserDetailsService userDetailsService;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        // Newer Spring Security versions require the UserDetailsService
        // passed directly into the constructor (no-arg + setter is removed).
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**", "/api/categories/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/products/**", "/api/categories/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/products/**", "/api/categories/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/products/**", "/api/categories/**").hasRole("ADMIN")
                .requestMatchers("/api/orders/**").authenticated()
                .anyRequest().authenticated()
            )
            .authenticationProvider(authenticationProvider())
```

```
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Why this matters:

csrf().disable() is safe and standard here: CSRF protection exists for cookie/session-based auth, and this API is stateless JWT via the Authorization header, not cookies. **SessionCreationPolicy.STATELESS** tells Spring Security to never create or use an HTTP session — the core of what makes JWT auth scale without server-side session storage. The authorizeHttpRequests rules are evaluated top to bottom, first match wins: auth endpoints are public, GET on products/categories is public browsing, write operations on the catalog require ADMIN, order endpoints require any authenticated user, and everything else defaults to "must be logged in" as a secure-by-default fallback. **hasRole("ADMIN")** (not "ROLE_ADMIN") is important — Spring Security auto-prepends ROLE_ when using hasRole(), matching the Role.ROLE_ADMIN enum used in UserPrincipal. Finally, addFilterBefore wires the custom JwtAuthenticationFilter to run before Spring's own default filter in the chain.

Step 7 — Auth Controller & Service

The first fully testable endpoints: register and login. Verified end-to-end with Postman.

service/AuthService.java

```
package com.shopsphere.shopsphere.service;

import com.shopsphere.shopsphere.dto.request.LoginRequest;
import com.shopsphere.shopsphere.dto.request.RegisterRequest;
import com.shopsphere.shopsphere.dto.response.AuthResponse;
import com.shopsphere.shopsphere.entity.Role;
import com.shopsphere.shopsphere.entity.User;
import com.shopsphere.shopsphere.repository.UserRepository;
import com.shopsphere.shopsphere.security.JwtService;
import com.shopsphere.shopsphere.security.UserPrincipal;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.HashSet;
import java.util.Set;

@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;

    @Transactional
    public AuthResponse register(RegisterRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new IllegalArgumentException("Email already registered: " + request.getEmail());
        }

        Set<Role> roles = new HashSet<>();
        roles.add(Role.ROLE_CUSTOMER);

        User user = User.builder()
                .fullName(request.getFullName())
                .email(request.getEmail())
                .password(passwordEncoder.encode(request.getPassword()))
                .roles(roles)
                .enabled(true)
                .build();

        userRepository.save(user);

        UserPrincipal principal = new UserPrincipal(user);
        String token = jwtService.generateToken(principal);

        return AuthResponse.builder()
                .accessToken(token)
                .tokenType("Bearer")
                .expiresInMs(jwtService.getExpirationMs())
                .build();
    }

    public AuthResponse login(LoginRequest request) {
        authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
        );

        User user = userRepository.findByEmail(request.getEmail())
                .orElseThrow(() -> new IllegalStateException("User not found after authentication"));
        UserPrincipal principal = new UserPrincipal(user);
```

```
        String token = jwtService.generateToken(principal);

        return AuthResponse.builder()
                .accessToken(token)
                .tokenType("Bearer")
                .expiresInMs(jwtService.getExpirationMs())
                .build();
    }
}
```

controller/AuthController.java

```
package com.shopsphere.shopsphere.controller;

import com.shopsphere.shopsphere.dto.request.LoginRequest;
import com.shopsphere.shopsphere.dto.request.RegisterRequest;
import com.shopsphere.shopsphere.dto.response.AuthResponse;
import com.shopsphere.shopsphere.service.AuthService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
        AuthResponse response = authService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        AuthResponse response = authService.login(request);
        return ResponseEntity.ok(response);
    }
}
```

Why this matters:

Every new registration is hard-coded to ROLE_CUSTOMER — nobody can self-assign ADMIN through the API. This is a deliberate security decision: admin accounts should only ever be created via a trusted process (a DB seed, or promotion by an existing admin), never open self-registration. **passwordEncoder.encode()** is where BCrypt hashing actually happens, right before saving — the raw password never touches the database. **login() delegates to authenticationManager.authenticate()** rather than manually comparing passwords; internally this uses the DaoAuthenticationProvider configured in Step 6 (CustomUserDetailsService + BCryptPasswordEncoder), and throws automatically on bad credentials. 201 Created for register vs 200 OK for login follows correct REST convention (register creates a new resource, login doesn't).

Step 8 — Category & Product REST APIs

Full CRUD for the product catalog, with pagination, category filtering, and name search on products. Access control is entirely centralized in SecurityConfig — these controllers carry zero security annotations.

service/CategoryService.java

```
package com.shopsphere.shopsphere.service;

import com.shopsphere.shopsphere.dto.request.CategoryRequest;
import com.shopsphere.shopsphere.dto.response.CategoryResponse;
import com.shopsphere.shopsphere.entity.Category;
import com.shopsphere.shopsphere.repository.CategoryRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@RequiredArgsConstructor
public class CategoryService {

    private final CategoryRepository categoryRepository;

    @Transactional
    public CategoryResponse create(CategoryRequest request) {
        if (categoryRepository.existsByNameIgnoreCase(request.getName())) {
            throw new IllegalArgumentException("Category already exists: " + request.getName());
        }

        Category category = Category.builder()
                .name(request.getName())
                .description(request.getDescription())
                .build();

        Category saved = categoryRepository.save(category);
        return toResponse(saved);
    }

    public List<CategoryResponse> getAll() {
        return categoryRepository.findAll().stream()
                .map(this::toResponse)
                .toList();
    }

    public CategoryResponse getById(Long id) {
        Category category = categoryRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Category not found: " + id));
        return toResponse(category);
    }

    @Transactional
    public CategoryResponse update(Long id, CategoryRequest request) {
        Category category = categoryRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Category not found: " + id));

        category.setName(request.getName());
        category.setDescription(request.getDescription());

        Category saved = categoryRepository.save(category);
        return toResponse(saved);
    }

    @Transactional
    public void delete(Long id) {
        if (!categoryRepository.existsById(id)) {
            throw new IllegalArgumentException("Category not found: " + id);
        }
        categoryRepository.deleteById(id);
    }
```

```
    private CategoryResponse toResponse(Category category) {
        return CategoryResponse.builder()
                .id(category.getId())
                .name(category.getName())
                .description(category.getDescription())
                .build();
    }
}
```

controller/CategoryController.java

```
package com.shopsphere.shopsphere.controller;

import com.shopsphere.shopsphere.dto.request.CategoryRequest;
import com.shopsphere.shopsphere.dto.response.CategoryResponse;
import com.shopsphere.shopsphere.service.CategoryService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/categories")
@RequiredArgsConstructor
public class CategoryController {

    private final CategoryService categoryService;

    @PostMapping
    public ResponseEntity<CategoryResponse> create(@Valid @RequestBody CategoryRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(categoryService.create(request));
    }

    @GetMapping
    public ResponseEntity<List<CategoryResponse>> getAll() {
        return ResponseEntity.ok(categoryService.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<CategoryResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(categoryService.getById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<CategoryResponse> update(@PathVariable Long id, @Valid @RequestBody CategoryRequest request) {
        return ResponseEntity.ok(categoryService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        categoryService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

service/ProductService.java

```
package com.shopsphere.shopsphere.service;

import com.shopsphere.shopsphere.dto.request.ProductRequest;
import com.shopsphere.shopsphere.dto.response.ProductResponse;
import com.shopsphere.shopsphere.entity.Category;
import com.shopsphere.shopsphere.entity.Product;
import com.shopsphere.shopsphere.repository.CategoryRepository;
import com.shopsphere.shopsphere.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
@Service
```

```
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;

    @Transactional
    public ProductResponse create(ProductRequest request) {
        Category category = categoryRepository.findById(request.getCategoryId())
                .orElseThrow(() -> new IllegalArgumentException("Category not found: " + request.getCategoryId()));

        Product product = Product.builder()
                .name(request.getName())
                .description(request.getDescription())
                .price(request.getPrice())
                .stockQuantity(request.getStockQuantity())
                .category(category)
                .build();

        Product saved = productRepository.save(product);
        return toResponse(saved);
    }

    public Page<ProductResponse> getAll(Pageable pageable) {
        return productRepository.findAll(pageable).map(this::toResponse);
    }

    public Page<ProductResponse> getByCategory(Long categoryId, Pageable pageable) {
        return productRepository.findByCategoryId(categoryId, pageable).map(this::toResponse);
    }

    public Page<ProductResponse> search(String name, Pageable pageable) {
        return productRepository.findByNameContainingIgnoreCase(name, pageable).map(this::toResponse);
    }

    public ProductResponse getById(Long id) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found: " + id));
        return toResponse(product);
    }

    @Transactional
    public ProductResponse update(Long id, ProductRequest request) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found: " + id));

        Category category = categoryRepository.findById(request.getCategoryId())
                .orElseThrow(() -> new IllegalArgumentException("Category not found: " + request.getCategoryId()));

        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStockQuantity(request.getStockQuantity());
        product.setCategory(category);

        Product saved = productRepository.save(product);
        return toResponse(saved);
    }

    @Transactional
    public void delete(Long id) {
        if (!productRepository.existsById(id)) {
            throw new IllegalArgumentException("Product not found: " + id);
        }
        productRepository.deleteById(id);
    }

    private ProductResponse toResponse(Product product) {
        return ProductResponse.builder()
                .id(product.getId())
                .name(product.getName())
                .description(product.getDescription())
                .price(product.getPrice())
```

```
                .stockQuantity(product.getStockQuantity())
                .categoryId(product.getCategory().getId())
                .categoryName(product.getCategory().getName())
                .build();
    }
}
```

controller/ProductController.java

```
package com.shopsphere.shopsphere.controller;

import com.shopsphere.shopsphere.dto.request.ProductRequest;
import com.shopsphere.shopsphere.dto.response.ProductResponse;
import com.shopsphere.shopsphere.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @PostMapping
    public ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(productService.create(request));
    }

    @GetMapping
    public ResponseEntity<Page<ProductResponse>> getAll(
            @RequestParam(required = false) Long categoryId,
            @RequestParam(required = false) String name,
            Pageable pageable) {

        if (categoryId != null) {
            return ResponseEntity.ok(productService.getByCategory(categoryId, pageable));
        }
        if (name != null && !name.isBlank()) {
            return ResponseEntity.ok(productService.search(name, pageable));
        }
        return ResponseEntity.ok(productService.getAll(pageable));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductResponse> update(@PathVariable Long id, @Valid @RequestBody ProductRequest request) {
        return ResponseEntity.ok(productService.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Why this matters:

The toResponse() private helper is simple manual entity→DTO mapping — perfectly fine and arguably clearer than a mapping library (like MapStruct) at this scale; that's worth reaching for once there are dozens of entities or deeply nested mapping logic. Neither controller has a single @PreAuthorize annotation — all access rules live centrally in SecurityConfig from Step 6. This is a deliberate architectural choice: centralizing security rules makes them easy to

audit in one place, versus scattering annotations across every controller method (both are valid, real-world approaches). The Pageable parameter in ProductController.getAll is populated automatically by Spring from query params like ?page=0&size=10&sort=price,desc with zero extra code, and the optional categoryId/name params let one endpoint serve three query patterns cleanly.

Verified Test Results (Postman)

Every endpoint built so far has been manually tested end-to-end against a live MySQL database, not just compiled. Results:

Test	Endpoint	Result
Register new user	POST /api/auth/register	201 Created — JWT issued
Login (correct credentials)	POST /api/auth/login	200 OK — fresh JWT issued
Login (wrong password)	POST /api/auth/login	403 Forbidden — correctly rejected
Create category, no token	POST /api/categories	403 Forbidden — blocked pre-routing
Create category, CUSTOMER token	POST /api/categories	403 Forbidden — role enforced
Create category, ADMIN token	POST /api/categories	201 Created — role allowed
Create product, ADMIN token	POST /api/products	201 Created — linked to category
List products, no token	GET /api/products	200 OK — public read confirmed

Key confirmation from this testing: role changes only take effect on a freshly issued token. After promoting a user to ROLE_ADMIN directly in the database, the old JWT (issued before the promotion) still carried the outdated role claims — a fresh login was required to get a token reflecting the new role. This is expected, stateless-JWT behavior and a good thing to be able to explain: the token is a snapshot of identity/roles at issuance time, not a live pointer back to the database.

What's next: Step 9 — Order Placement

The most involved piece of the whole project: checking stock availability, decrementing it atomically, snapshotting purchase price into OrderItem, calculating order totals, and using the @Version optimistic lock plus the findByIdForUpdate pessimistic lock (both built in Steps 2 and 3 but unused until now) to prevent two simultaneous orders from overselling the same product.